MuLearn CET

TECH TEAM RECRUITMENT

Technical Challenges

How This Works

Pick **ONE** of the four challenges that follow, based on what interests you.

We are not looking for a perfect product. We want to see that you can research a problem, understand the concepts, make decisions and build something that works.

Every challenge has two parts:

- **Must-have:** the core of the task. A working version of this is a complete submission.
- **Bonus:** optional extras. Try these only after the must-haves work.

Finishing the must-haves well is better than half-finishing everything.

What to Submit

1. A **GitHub repository** with your complete source code.
2. A **README** explaining what the project does and how to set it up and run it.
3. A short **demo video (2–3 minutes)** showing it working. A deployed link is a bonus.

Shortlisted candidates will have a short **viva** where you explain your code and your decisions.

Guidelines

- You may use AI tools, libraries, frameworks and documentation.
- You **must understand and be able to explain** everything you submit. Copying a complete project without understanding it is not acceptable.
- Be ready to justify your technical choices.
- Keep the scope realistic. **Get a simple version working first, then improve it.**

Task 01: Peer-to-Peer Chat

Domain: Networking

Objective

Build a simple chat app where two devices talk to each other directly (peer-to-peer) instead of sending every message through a central chat server.

Must-have

- Two browsers connect and exchange real-time text messages using a **WebRTC data channel**.
- A small signaling server (Node.js + WebSockets) is allowed, only to help the two peers find each other.
- Show the connection status (connecting, connected, disconnected).

Bonus

- Send files between peers.
- Support more than two peers (group chat).
- Handle a peer disconnecting and reconnecting.
- Work across different networks (hint: look up STUN servers).

Areas to Explore

WebRTC, WebSockets, Node.js, signaling.

Evaluation Focus

Do you understand how two peers find and connect to each other? Does the chat work reliably between two devices?

Task 02: Loops, a Puzzle Platformer

Domain: Game Development

Objective

Build a small 2D puzzle platformer where things run in repeating cycles (loops). The player observes these loops and uses them to solve puzzles.

Must-have

- Player movement, jumping and collision detection.
- At least **one looping element** (moving platform, patrolling enemy or timed laser).
- At least **one player loop mechanic**. Example: the player records a short movement path and a "ghost" replays it on repeat to hold a switch or open a door.
- A switch or pressure plate, and a door.
- **2–3 levels** with a win condition, a fail condition and a restart option.

Bonus

- Several loops running together at different speeds.
- Letting the player edit, start or stop loops.
- More levels that get progressively harder.
- Sound and visual polish.

Technology

Any engine or framework (Godot, Unity, GameMaker, Pygame, etc.).

Evaluation Focus

The loop mechanic, game logic and level design. Graphics matter least.

Task 03: Beyond Sentiment, Detecting Emotion in Text

Domain: AI and NLP

Objective

Build a system that reads a sentence and predicts the emotion behind it, going beyond plain positive/negative classification.

Must-have

- **Sentiment** classification (Positive, Negative, Neutral).
- **Emotion** classification (e.g. joy, sadness, anger, fear, surprise) using a public dataset.
- Use a classic approach (TF-IDF + Logistic Regression or Naive Bayes) **or** a pretrained Hugging Face model.
- Show a **confidence score** with every prediction.
- Evaluate your model (accuracy, plus a short note on where it fails).
- A simple way to try it: command-line script, notebook or small web page.

Bonus

- Sarcasm detection.
- Explanations (which words influenced the prediction).
- Using surrounding context to improve interpretation.
- Fine-tuning a BERT-style model.

Example Inputs

Input	Possible Interpretation
"Yeah, that was just great..."	Sarcasm / Disappointment
"I guess I'm used to it now."	Resignation
"Do whatever you want."	Possible Frustration
"Wow, another deadline. Amazing!"	Sarcasm
"It's fine. Don't worry about me."	Possible Concealed Sadness

Important

Your system should express **uncertainty**. It predicts likely emotions; it does not claim to know what a person truly means.

Areas to Explore

NLP basics, TF-IDF, Logistic Regression, Naive Bayes, Hugging Face, PyTorch.

Task 04: Project Management Web App

Domain: Web Development

Objective

Build a small project management app inspired by Trello. Users sign in, create projects and tasks, and move tasks across a Kanban board.

Must-have

1. **Authentication:** register and log in, with passwords stored hashed.
2. **Projects:** create, edit and delete projects (title, description, deadline).
3. **Tasks:** title, description, assigned user, priority (Low, Medium, High, Critical), deadline and status.
4. **Kanban board:** columns To Do, In Progress, Review, Completed. Drag and drop tasks between columns, with changes saved in the database (still there after refresh).
5. **Dashboard:** total, completed, pending and overdue task counts, plus at least one chart.
6. **UI/UX:** clean, consistent, easy to navigate, and usable on both desktop and mobile.

Bonus

- Roles (admin / member) with different permissions.
- Comments on tasks and notifications on assignment.
- Search and filter by priority or assignee.
- Real-time updates (Socket.IO), file attachments, deployment.

Suggested Stack (alternatives allowed)

Layer	Suggestion
Frontend	React / Next.js
Backend	Node.js / Express
Database	MongoDB / PostgreSQL
Styling	Tailwind CSS
Auth	JWT / Sessions
Charts	Recharts